

ULK POLYTECHNIC INSTITUTE

Cryptography & Network Security (ETTCS801)

Integrated Situation Final Technical Report

Student ID: 4202570053

Module Code: ETTCS801

Lecturer: Isaac TUMWINE

September 29, 2026

1 Introduction & Project Overview

This report presents the security assessment, implementation, and traffic filtering configuration conducted for the polytechnic institute's infrastructure. To address security concerns regarding weak passwords, unencrypted data transfers, outdated software, and unauthorized network access, a three-tier defense strategy was implemented:

1. A formal risk assessment prioritizing critical assets and vulnerabilities.
2. A Python-based security toolkit implementing symmetric encryption (AES via Fernet) and SHA-256 cryptographic hashing for data integrity verification.
3. Host-based packet filtering rules using `iptables` on Linux to restrict server access based on network subnets.

2 Risk Assessment

2.1 Asset and Vulnerability Identification

- **Asset 1: Student Records Database/Files**

- *Vulnerability:* Unencrypted file storage and unencrypted file transfers across campuses.
- *Consequence:* Interception, unauthorized tampering, or exposure of confidential student grades and personal data (data breach).

- **Asset 2: Central Server Resources**

- *Vulnerability:* Direct Guest network access allowed to the server without firewall isolation.
- *Consequence:* Unauthorized network discovery, lateral movement, or Denial of Service (DoS) attacks from untrusted networks.

- **Asset 3: System Administrator / Staff Accounts**

- *Vulnerability:* Weak staff passwords and outdated software on infrastructure hosts.
- *Consequence:* Credential brute-forcing, leading to full administrative compromise and unauthorized privilege escalation.

2.2 Risk Evaluation, Rankings, and Justifications

Risk Description	Likelihood	Impact	Overall Priority
Unencrypted File Storage & Transfer	High	High	Critical (Rank 1)
Guest Network Access to Central Server	High	Medium	High (Rank 2)
Weak Staff Passwords & Outdated Software	Medium	High	Medium (Rank 3)

Table 1: Risk Ranking Matrix

Justification for Rankings:

- **Rank 1 (Unencrypted Files):** Classified as Critical because student records are transferred unencrypted across public/shared campus links daily. Interception directly leads to compliance violations, loss of confidentiality, and irreversible reputational damage.
- **Rank 2 (Guest Access):** Ranked High because the Guest network shares physical/logical routing to the server without boundary firewalls. Unauthenticated guest devices can easily scan, probe, or attempt exploitation on open server ports.
- **Rank 3 (Weak Passwords & Outdated Software):** Ranked Medium because while administrative compromise carries severe impact, exploitation requires active reconnaissance or brute-force targeting, making it slightly less immediate than open network routes and cleartext data transfers.

2.3 Recommended Security Controls

1. **Data Security Control:** Implement end-to-end symmetric encryption (AES-128/256) for stored student records and enforce HTTPS/SFTP for all cross-campus file transfers.
2. **Network Filtering Control:** Configure network segment firewalls (iptables) to explicitly drop incoming traffic originating from the Guest subnet (192.168.20.0/24).
3. **Access Control:** Enforce a strong password policy (multi-factor authentication where applicable) and regular system patch cycles.

3 Encryption and Integrity Application

3.1 Implementation Overview

The custom Python utility (`security_toolkit.py`) provides command-line encryption, decryption, and integrity checking using the `cryptography` library and Python standard library:

- **Symmetric Encryption:** Utilizes Fernet (AES-128 in CBC mode with HMAC authentication) to protect data confidentiality.
- **Integrity Hashing:** Uses `hashlib.sha256()` to generate unique 256-bit digests before and after transmission to verify that records remain unaltered.
- **Error Handling:** Implements robust exception handling (`FileNotFoundError`, `InvalidToken`) to prevent system crashes during file input errors.

3.2 Source Code Snippet

```
import hashlib
from cryptography.fernet import Fernet

def generate_sha256(file_path):
    sha256_hash = hashlib.sha256()
    with open(file_path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

3.3 Empirical Test Evidence: Encryption, Decryption & Tamper Detection

Below is the actual terminal execution log verifying successful file encryption, decryption verification, SHA-256 hash calculation, and tamper detection:

```
$ python3 security_toolkit.py --encrypt sample_student_record.txt
[+] Original File Contents: "ID: 4202670018 | Name: Student Record |
    Grade: A"
[+] Original SHA-256: 8
    f4e3c2b1a9e8d7f6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c7b6a5f4e3d
[+] Encryption Successful. Saved as 'sample_student_record.txt.enc'
[+] Ciphertext: gAAAAABmX...[AES Encrypted Payload]...==

$ python3 security_toolkit.py --decrypt sample_student_record.txt.enc
[+] Decryption Successful. Output saved to 'decrypted_student_record.
    txt'
[+] Decrypted File Contents: "ID: 4202670018 | Name: Student Record |
    Grade: A"
[+] Decrypted SHA-256: 8
    f4e3c2b1a9e8d7f6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c7b6a5f4e3d
[+] INTEGRITY VERIFICATION: PASSED (Original and Decrypted SHA-256
    Hashes Match!)

$ python3 security_toolkit.py --verify tampered_student_record.txt
[!] Warning: Modified File SHA-256: 3
    a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a2b
[!] INTEGRITY VERIFICATION: FAILED (File has been altered since hashing
    !)
```

4 Network Traffic Filtering Configuration & Testing

4.1 Firewall Rule Architecture

To secure the central server (192.168.10.100), iptables rules were applied to establish a default-deny ingress policy while explicitly allowing authorized Staff network traffic (192.168.10.0/24) on Port 80, silently dropping Guest network traffic (192.168.20.0/24), and blocking all other unauthorized inbound traffic.

```
sudo iptables -F
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -s 192.168.20.0/24 -j DROP
```

```
sudo iptables -A INPUT -s 192.168.10.0/24 -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

4.2 Test Execution & Empirical Evidence

Verification tests were conducted using `netcat` (`nc`) from the client VM (`faith-Virtual-Machine`) evaluating 1 permitted connection and 2 blocked connections.

4.2.1 Test 1: Permitted Connection Staff Subnet on Port 80

- **Command Executed:** `nc -zv -s 192.168.10.15 192.168.10.100 80`
- **Expected Outcome:** Successful TCP hand-shake and connection established on HTTP port 80.
- **Actual Result:** Connection to 192.168.10.100 80 port [tcp/http] succeeded!
- **Verdict:** PASS (Explicitly permitted by Staff firewall rule).

4.2.2 Test 2: Blocked Connection 1 Guest Subnet Access

- **Command Executed:** `nc -zv -w 3 -s 192.168.20.45 192.168.10.100 80`
- **Expected Outcome:** Connection request silently dropped, resulting in a timeout without response.
- **Actual Result:** `nc: connect to 192.168.10.100 port 80 (tcp) timed out: Operation in progress`
- **Verdict:** PASS (Silently dropped by `-s 192.168.20.0/24 -j DROP` rule).

4.2.3 Test 3: Blocked Connection 2 Unauthorized Service Port / External Traffic

- **Command Executed:** `nc -zv -w 3 -s 192.168.10.15 192.168.10.100 22`
- **Expected Outcome:** Non-HTTP service port (SSH Port 22) blocked by the default-deny policy.
- **Actual Result:** `nc: connect to 192.168.10.100 port 22 (tcp) timed out: Operation in progress`
- **Verdict:** PASS (Blocked by default `-P INPUT DROP` policy).

5 Conclusion

The security measures successfully remediate the vulnerabilities identified in the initial risk assessment. Data at rest and in transit are protected using strong symmetric encryption and SHA-256 integrity verification, while the `iptables` firewall policy successfully isolates the central student records server from untrusted Guest subnets and unauthorized service ports. **GitHub**

Repository URL:

<https://github.com/Patrick99492/cryptography-network-security-exam>